

Day 1: Setup, Python Refresher, and Your First Circuit

SC Quantathon v3 Bootcamp
Clemson Quantum Club

These notes follow the Day 1 notebook section by section. The notebook is where you run things; these pages give the explanations, the equations, and the figures behind each step, so that either one can be read on its own.

1 Setup

The notebook runs in two places. Locally, the repository's `requirements.txt` pins Qiskit 2.5, the Aer simulator, and the IBM runtime client into a conda environment. On qBraid Lab those packages are preinstalled. Every cell behaves the same on both.

Two accounts matter this week. The IBM Quantum Open Plan gives ten minutes of QPU time per 28-day rolling window on real superconducting processors, and the notebook's section 2 stores its API key on disk once under the name `scqv3` so that no later notebook needs the key in a cell. The qBraid account is where the hackathon credits live; Day 3 covers that platform.

2 Python essentials

Everything in the bootcamp is built from a small set of Python pieces. Table 1 lists the ones that appear in every later notebook, with the reason each one matters here.

Construct	Example	Where it shows up
list	<code>["00", "01", "10", "11"]</code>	the possible outcomes of a circuit
dictionary	<code>{"00": 503, "11": 497}</code>	every measurement result Qiskit returns
<code>.get(key, 0)</code>	<code>counts.get("01", 0)</code>	reading an outcome that may not have occurred
function	<code>def probability_of_zero(...)</code>	the exercises from Day 2 on
f-string	<code>f"{p:.3f}"</code>	printing probabilities to three decimals
numpy array	<code>np.array([1, 0])</code>	qubit states and gate matrices
<code>@</code>	<code>U @ psi</code>	applying a gate to a state
<code>.conj().T</code>	<code>U.conj().T</code>	the conjugate transpose $U^\dagger$
<code>np.allclose</code>	<code>np.allclose(A, B)</code>	comparing floating-point arrays

Table 1: The Python and numpy constructs the bootcamp uses.

2.1 Complex numbers

A complex number is $z = x + iy$ with $i^2 = -1$; Python writes i as `1j`. Three quantities are used constantly from Day 2 onward. The magnitude, the conjugate, and their product:

$$|z| = \sqrt{x^2 + y^2}, \quad \bar{z} = x - iy, \quad z\bar{z} = x^2 + y^2 = |z|^2. \quad (1)$$

Any complex number of magnitude one can be written as a point on the unit circle,

$$e^{i\phi} = \cos \phi + i \sin \phi, \quad |e^{i\phi}| = 1 \text{ for every real } \phi, \quad (2)$$

which is what `np.exp(1j * phi)` computes. In numpy, `abs(z)`, `np.conj(z)`, and `np.angle(z)` return $|z|$, $\bar{z}$, and ϕ .

2.2 Vectors and matrices

A vector v with complex entries is *normalized* when

$$\sum_i |v_i|^2 = 1. \quad (3)$$

Dividing any nonzero vector by $\sqrt{\sum_i |v_i|^2}$ produces a normalized one. Matrices act on vectors by the product $M @ v$, and the conjugate transpose $M^\dagger$ is `M.conj().T`. The notebook's example is a quarter turn of the plane,

$$R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \quad R \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad R^T R = I, \quad (4)$$

so undoing the rotation is the same as applying the transpose. Quantum gates on Day 2 will be exactly this kind of matrix, with $R^\dagger R = I$ in place of $R^T R = I$.

One habit matters from the first cell on: never test floating-point results with `==`. In binary floating point $0.1 + 0.2$ is not 0.3 but 0.30000000000000004 , and a normalized vector computed by dividing by a square root has a norm of 1 only to about 16 digits. `np.allclose(a, b)` compares two arrays to within a tolerance (by default 10^{-8} absolute plus 10^{-5} relative) and is what every Checkpoint in the bootcamp uses; `np.isclose` does the same for single numbers.

3 Bits and gates

A bit is always exactly one of two values, 0 or 1. Reading it does not change it. A classical gate is a lookup table from input bits to output bits; Table 2 shows the two simplest ones.

a	NOT a	a	b	$a \wedge b$	a	b	$a \oplus b$
0	1	0	0	0	0	0	0
0	1	0	1	0	0	1	1
1	0	1	0	0	1	0	1
1	0	1	1	1	1	1	0

Table 2: NOT, AND, and XOR as lookup tables.

NOT is *reversible*: applying it twice returns the input. AND is not. Given the output 0, three different input pairs could have produced it, so the inputs are lost. Most classical gates discard information this way.

XOR alone is also irreversible, since $(0, 1)$ and $(1, 0)$ both give 1. But the two-output map

$$(a, b) \mapsto (a, a \oplus b) \quad (5)$$

sends the four inputs to four distinct outputs, so it can be undone. In words: keep the first bit, and flip the second bit when the first is 1. Every quantum gate is reversible, and Eq. (5) is the truth table of the two-qubit gate called CNOT that appears on Day 4.

There is a counting argument behind this. A gate on n bits has 2^n possible inputs. It is reversible when it sends them to 2^n distinct outputs; with as many output bits as input bits, that means its table is a *permutation* of the 2^n bitstrings. NOT permutes two strings, Eq. (5) permutes four (writing the pair as ab , it swaps 10 and 11 and fixes the rest), and AND sends four inputs onto two outputs and so cannot be a permutation. The quantum rule of Day 2, that a gate is a unitary matrix, is the same requirement stated for amplitudes instead of bits: a unitary sends distinct states to distinct states and has an inverse.

4 Qubits, in words

Until it is measured, a qubit is a weighted blend of 0 and 1 called a *superposition*. Measuring it forces one of the two answers and destroys the blend. Because a single measurement returns a single bit, a quantum program is read by running it many times and looking at the histogram of outcomes.

A qubit also carries a *phase*, a sign (more generally an angle) attached to each of the two possibilities. Signs can cancel, and that cancellation, called *interference*, is what a classical coin cannot do. Day 2 writes the phase down as a complex number.

Three ingredients separate a quantum computer from a classical one:

- **Superposition.** One qubit holds a blend of 0 and 1; n qubits hold a blend of all 2^n bitstrings.
- **Entanglement.** Qubits whose outcomes are linked, so that measuring one tells you the other.
- **Interference.** Phases arranged so that wrong answers cancel and the right one adds up.

A quantum algorithm uses all three and then measures. Today's circuit uses only the first.

5 The quantum random number generator

5.1 The circuit

The smallest useful quantum program has one qubit, one gate, and one measurement (Figure 1). The qubit starts in the state written $|0\rangle$. The Hadamard gate H turns it into an equal superposition of $|0\rangle$ and $|1\rangle$, and the measurement returns 0 or 1:

$$|0\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \xrightarrow{\text{measure}} 0 \text{ or } 1, \quad \text{each with probability } \frac{1}{2}. \quad (6)$$

The factor $1/\sqrt{2}$ is there so the state is normalized in the sense of Eq. (3): the probabilities are the squared magnitudes of the coefficients, $|1/\sqrt{2}|^2 = 1/2$ each. Day 2 makes this precise.

In Qiskit the circuit is an object that operations are appended to, one per line:

```
qc = QuantumCircuit(1, 1)      # 1 qubit, 1 classical bit
qc.h(0)                        # Hadamard on qubit 0
qc.measure(0, 0)               # measure qubit 0 into classical bit 0
```

5.2 Running it

A simulator is a classical program that computes what the circuit would output. Running the circuit for a number of *shots* returns a dictionary from outcome to count:

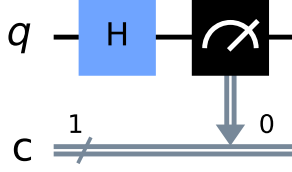


Figure 1: The one-qubit random number generator as the notebook draws it. Read left to right: the qubit q starts in $|0\rangle$, passes through H , and is measured into the classical bit c .

```
result = AerSimulator().run(qc, shots=1000).result()
counts = result.get_counts()           # e.g. {'0': 489, '1': 511}
```

Those two lines, together with the three above, are the pattern for every circuit in the bootcamp: build, run, read the counts. The notebook's run with 1000 shots gave 503 zeros and 497 ones (Figure 2); the simulator there is unseeded, so your own run will differ by a few counts.

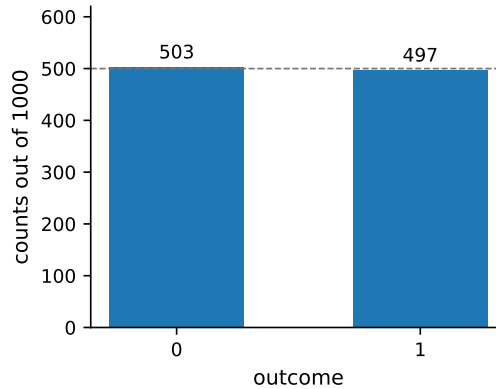


Figure 2: The circuit of Figure 1 on the Aer simulator, 1000 shots. The dashed line is the ideal 500.

5.3 Shots and statistics

Each shot is an independent draw that gives 0 with probability p . Over N shots the number of zeros k follows a binomial distribution with mean and variance

$$\mathbb{E}[k] = Np, \quad \text{Var}[k] = Np(1 - p), \quad (7)$$

so the count fluctuates by one standard deviation

$$\sigma = \sqrt{Np(1 - p)}. \quad (8)$$

For the fair coin of Eq. (6), $p = 1/2$, and with $N = 1000$ this is $\sigma = \sqrt{250} \approx 16$ counts. A result of 484 or 516 zeros is ordinary. A result of 400 is more than six standard deviations away and would mean the circuit is not what you think it is.

The natural estimate of the probability is $\hat{p} = k/N$, whose standard deviation is

$$\sigma_{\hat{p}} = \frac{\sigma}{N} = \sqrt{\frac{p(1-p)}{N}}. \quad (9)$$

Precision improves as $1/\sqrt{N}$: ten times more shots buys about three times more precision. Turned around, Eq. (9) says how many shots a target precision costs: $N = p(1-p)/\sigma_{\hat{p}}^2$, which for a fair coin is 2500 shots for $\sigma_{\hat{p}} = 0.01$ and 250 000 shots for 0.001. Every extra decimal place costs a hundred times more shots, and on a real processor shots are the currency, ten minutes of them per month on the Open Plan. For N of a few dozen or more the binomial distribution is close to a Gaussian, so the usual rule applies: about 68% of runs land within one σ of the mean, 95% within two, and 99.7% within three, and a six- σ result like 400 zeros has a probability below 10^{-8} . The notebook's section 7.3 gave $\hat{p} = 0.700, 0.460, 0.497, 0.502$ at $N = 10, 100, 1000, 10000$: the estimate wanders at small N (seven zeros in ten is a 1.3- σ event, nothing unusual) and settles toward $1/2$ as Eq. (9) says it should. Figure 3 shows those four runs against a seeded sweep over four decades of N .

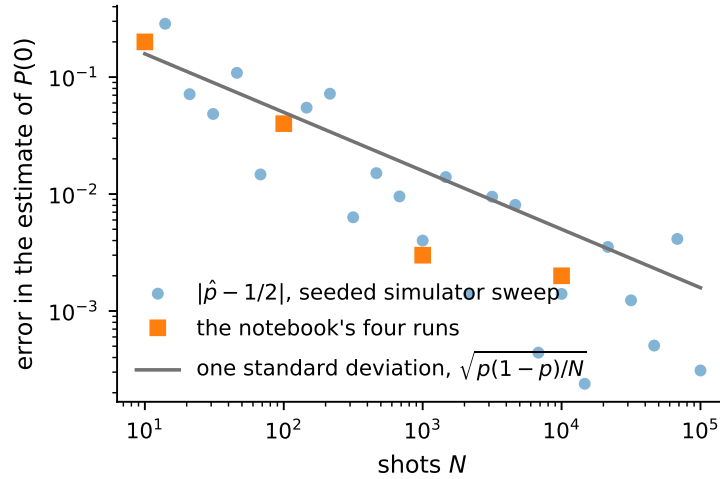


Figure 3: The error in the estimated $P(0)$ against the number of shots: the notebook's four runs (squares) and a seeded sweep (circles). The line is Eq. (9) with $p = 1/2$. Individual points scatter above and below it, as one standard deviation should.

5.4 Where the randomness comes from

Python's `random` module is a pseudorandom generator: seed it with the same number twice and it repeats the same sequence forever. The Aer simulator is also a classical program, so it accepts a seed too, and `AerSimulator(seed_simulator=1)` reproduces its outcomes exactly. The notebook demonstrates both.

Only on real hardware is the outcome undetermined before the measurement takes place. The simulator reproduces the *statistics* of Eq. (6) but not the physics.

The keyword `memory=True` asks the simulator to keep every individual shot in order, as a list of outcome strings, rather than only the totals. Joining 32 such outcomes gives a 32-character bitstring, and `int(bits, 2)` converts it to an integer between 0 and $2^{32} - 1$; the notebook's run produced 0110011111111010011000000011101, which is 1 744 646 173. Because each bit is an independent fair coin, every one of the 2^{32} integers is equally likely, with mean $2^{31} \approx 2.1 \times 10^9$; the string has no

structure that distinguishes it from `random.getrandbits(32)`, and no statistical test can tell the two sources apart. What differs is only the origin: one is a formula seeded by a number, the other, on hardware, is a measurement whose outcome did not exist before it was made. Removing the H gate, the notebook’s last exercise, turns the circuit into one that always reads 0, which shows that the randomness comes from the superposition and not from the measurement itself.

6 The same circuit on real hardware

A real quantum processor is a chip with a fixed set of native gates. Two steps are added to the code. The *transpiler* rewrites the circuit into those native gates; on IBM’s superconducting chips the Hadamard is not native and becomes $R_z(\pi/2)\sqrt{X}R_z(\pi/2)$, three native gates that equal H up to a global phase (`Operator.equiv` confirms it), which Day 5 explains. The $\sqrt{X}$ is one calibrated microwave pulse; the R_z rotations are bookkeeping in the control electronics and cost no time at all. Then a *Sampler* submits the rewritten circuit to the machine’s queue:

```
pm = generate_preset_pass_manager(optimization_level=1, backend=backend)
qc_isa = pm.run(qc)
job = Sampler(mode=backend).run([qc_isa], shots=1000)
counts = job.result()[0].data.c.get_counts()
```

Three ways of producing one random bit appear in the notebook: a classical coin from a seeded pseudorandom generator (section 6.2), the circuit on a simulator (section 7.2), and the same circuit on a quantum processor (this section). The processor run is the live 1000-shot job on `ibm_kingston` recorded in the solutions notebook. Table 3 lists the three with the distance of the count of zeros from 500 in units of the $\sigma \approx 16$ of Eq. (8); Figure 4 draws them side by side.

source	count of 0	count of 1	count of 0 – 500 / σ
classical coin, <code>rng.integers</code>	484	516	1.0
simulator, <code>AerSimulator</code>	503	497	0.2
quantum processor, <code>ibm_kingston</code>	295	705	13.0

Table 3: One random bit, 1000 draws each, three ways.

The coin and the simulator sit within one standard deviation of 500/500, because both are exact samples from a fair distribution. They differ from each other only in what they are simulating: the coin is `rng.integers`, the simulator is the linear algebra of Section 5. Neither is physically random; both repeat given the same seed.

The hardware row is the one to look at twice. A run a couple of standard deviations from 500 would be ordinary: for a Gaussian the two-sided probability of a $2\text{-}\sigma$ deviation is 0.046, so a fair coin does that about once in twenty runs, and one such run is no evidence of bias. The `ibm_kingston` run is another matter: 295 zeros is thirteen standard deviations from 500, and no amount of shot noise produces that. The circuit ran on physical qubit 0 of the chip (the transpiler at optimization level 1 takes the first qubit rather than searching for a good one), and that qubit’s calibration at the time was unremarkable: a readout error of 1.9% in each direction, a $\sqrt{X}$ error of 2×10^{-4} , a T_1 of 370 μs . None of those predicts a 70/30 split. What did was found later the same evening, when the Day 2 notebook ran on the same qubit: a prepared $|0\rangle$ was read as 1 in 55% of shots and a prepared $|1\rangle$ as 1 in 85%. A fair 50/50 state seen through that readout is reported as $\frac{1}{2}(0.55 + 0.85) = 70\%$ ones, which is what this run shows. The qubit’s *readout error*, normally a percent or two, had drifted far from its calibration, which is a snapshot taken every few hours. The right response to

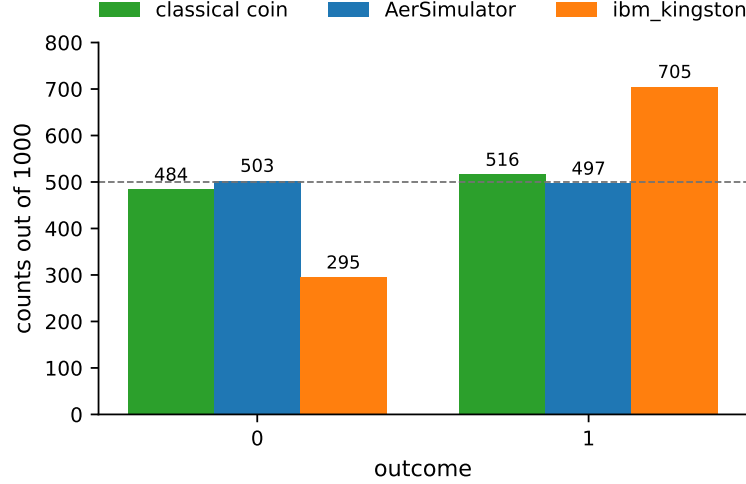


Figure 4: The three rows of Table 3. The dashed line is the ideal 500.

any single run that far from expectation is to rerun on a different qubit or machine before believing it. Day 5 reads the calibration table, chooses qubits with it, and measures readout error directly.

7 Summary

- A bit is one of two values; a classical gate is a lookup table, and most of them are irreversible. XOR with its first input kept, Eq. (5), is reversible and is the CNOT gate in disguise.
- A qubit is a superposition of 0 and 1 until measured, and carries a phase that lets possibilities cancel. A quantum program is read as a histogram over many shots.
- The random number generator, Eq. (6), is a Hadamard followed by a measurement. In Qiskit: build the circuit, run it on a backend with some number of shots, read the counts dictionary.
- Counts fluctuate by $\sigma = \sqrt{Np(1-p)}$ and the estimate of a probability improves as $1/\sqrt{N}$, Eqs. (8) and (9).
- Simulators are seeded classical programs. Only measurement on real hardware is undetermined in advance.
- Hardware adds a transpile step and a Sampler. Its counts carry readout error at the percent level, and occasionally a device error far larger than any statistics, so a single run is checked against σ before it is believed.

References

- [1] IBM Quantum, *Hello World*, tutorial. <https://quantum.cloud.ibm.com/docs/en/tutorials/hello-world>
- [2] J. Watrous, *Basics of Quantum Information*, IBM Quantum Learning. <https://quantum.cloud.ibm.com/learning/en/courses/basics-of-quantum-information>

- [3] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th anniversary ed. (Cambridge University Press, 2010), chapter 1.
- [4] Qiskit documentation. <https://quantum.cloud.ibm.com/docs>
- [5] The Python Tutorial. <https://docs.python.org/3/tutorial/>
- [6] NumPy quickstart. <https://numpy.org/doc/stable/user/quickstart.html>